RAPPORT PROJET
IShowCinema

LO3IN403 - OpsCi

Yanis H.
Antoine M.

Ressources du projet :
Vidéo de présentation
Dépôt GitLab

April 2026

Table des matières

1 Introduction et Objectifs **4**

2 Conception et Architecture **5**
2.1 Étude des architectures existantes 5
2.2 Étude des Design Patterns 5
2.3 Choix et Justification 6

3 Choix Techniques (Stack) **9**
3.1 Étude et Choix de la Stack Back-end 9
3.2 L'Environnement Virtuel (venv) 11
3.3 Étude et Choix de la Stack Front-end 11

4 Données et API **13**
4.1 Comprendre les API HTTP 13
4.2 Formats de Données : Le choix du JSON 13
4.3 Autorisation des accès entre Front-end et Back-end : CORS . 14
4.4 Intégration d'une API réelle (TMDB) et rôle de Proxy 14
4.5 Choix de la source de données : API externe vs Dataset existant 15
4.6 Collecte de données : Export vs Web Scraping 15
4.7 Bonnes pratiques avancées en production 16

5 Stockage et Performance **17**
5.1 Persistance des données avec MongoDB 17
5.2 Optimisation et Cache avec Redis 17

6 Conteneurisation et Orchestration **19**
6.1 Isolation des services avec Docker 19
6.2 Orchestration et Haute Disponibilité avec Kubernetes 19

7 CI/CD et Reproductibilité **21**
7.1 L'Infrastructure as Code (IaC) 21
7.2 Automatisation avec une Pipeline CI/CD 21
7.3 Scripts de lancement 22

8 Conclusion, Difficultés et Perspectives **23**
8.1 Bilan du projet 23
8.2 Difficultés techniques rencontrées 23

8.3 Conclusion générale 23

1 Introduction et Objectifs

Le projet IShowCinema s'inscrit dans une démarche de développement "Fullstack et Ops". L'objectif est de concevoir une plateforme de catalogue de films dynamique, capable de consommer une API réelle (TMDB), de persister des données (MongoDB) et d'optimiser les performances via un cache (Redis). L'enjeu majeur est l'orchestration de cette architecture via Kubernetes et l'automatisation via une CI/CD.

2 Conception et Architecture

2.1 Étude des architectures existantes

Avant de débuter l'implémentation, nous avons étudié plusieurs modèles :

— **Architecture Monolithique :** Un monolithe correspond à une application backend unique regroupant toute la logique métier. Facile à développer au début, mais difficile à faire évoluer ou à scaler.

— **Architecture en couches (Layered) :** Sépare la présentation, la logique métier et les données.

— **Architecture Micro-services :** Découpage de l'application en services indépendants communiquant par le réseau.

— **Architecture Client-Serveur :** Modèle où les responsabilités sont réparties entre un **client** qui initie les requêtes et un **serveur** qui traite les demandes et fournit les ressources. Cette séparation permet de centraliser les données et la logique métier tout en offrant une interface interactive à l'utilisateur.

2.2 Étude des Design Patterns

— **MVC (Model-View-Controller) :** Sépare les données, l'affichage et les actions utilisateur.

— **Singleton :** Garantit qu'une classe n'a qu'une seule instance (utile pour ne pas ouvrir 100 connexions à la base de données).

— **Repository :** Isole la logique de récupération des données.

— **Cache-Aside Pattern (Lazy Loading) :** Stratégie de mise en cache où l'application vérifie d'abord le cache avant d'interroger la source de vérité.

— **Facade Pattern :** Fournit une interface unifiée et simplifiée pour masquer la complexité d'un sous-système (une API externe).

— **Concurrency Pattern :** Gestion non-bloquante des entrées/sorties pour maximiser le débit des requêtes.

2.3 Choix et Justification

Après avoir étudié les différentes options, nous avons opté pour une **Architecture Client-Serveur distribuée**, découpant notre application en composants conteneurisés distincts.

Justification de l'Architecture

— **Le choix du Client-Serveur :** Contrairement à un monolithe (où le serveur génère directement les pages HTML), notre Front-end (React) agit comme un client qui communique avec notre Back-end (FastAPI) uniquement via une API REST en JSON.

— **Une architecture distribuée conteneurisée :** Bien que notre API backend ne soit pas fragmentée en plusieurs petits sous-services, notre architecture d'infrastructure est distribuée. Chaque composant (Client, API, Base de données, Cache) est conteneurisé indépendamment sous Docker et orchestré par Kubernetes.

— **Pourquoi nous n'avons pas retenu les autres options :** Nous avons écarté une architecture monolithique stricte côté backend. Bien que notre backend reste une application unique, nous avons préféré séparer le frontend et adopter une architecture distribuée conteneurisée pour faciliter le déploiement et la scalabilité.

Design Patterns retenus

Nous avons sélectionné les patterns suivants pour notre backend : Cache-Aside, Repository, Facade et Concurrency.

— **Cache-Aside Pattern :** Couplé à notre base Redis. Lorsqu'un utilisateur recherche un film, le backend cherche d'abord la réponse dans Redis. En cas de "Cache Miss" (donnée absente), le backend interroge TMDB, formate la réponse, la sauvegarde dans Redis avec une durée

de vie (TTL), puis la renvoie au client.

Justification : Ce pattern nous protège d'un blocage en servant instantanément les requêtes fréquentes (comme la page d'accueil des films populaires) depuis la mémoire vive.

— **Le Repository Pattern (côté Backend) :** Nous avons partiellement isolé la logique d'accès aux données via des fonctions intermédiaires, ce qui constitue une première approche du Repository Pattern.

Justification : Le Repository Pattern rend le code du backend beaucoup plus propre et maintenable. Si demain nous décidons de remplacer TMDB par une autre API, nous n'aurons qu'à modifier la couche "Repository" sans toucher à nos routes FastAPI.

— **Facade Pattern :** Notre backend FastAPI agit comme une Facade entre le client (React) et l'API externe (TMDB). Le front-end n'appelle jamais TMDB directement. Cela résout deux problèmes majeurs :
  — *Sécurité :* La clé secrète de l'API TMDB reste cachée côté serveur.

  — *Normalisation :* L'API TMDB renvoie des structures complexes. Notre Facade filtre, nettoie et aplatit ces données pour renvoyer au front-end un JSON standardisé et facile à consommer, réduisant ainsi la charge de calcul côté client.

— **Concurrency Pattern :** Nous utilisons un modèle de concurrence basé sur un pool de threads (ThreadPoolExecutor) afin de paralléliser les appels réseau.

Justification : Lorsqu'un utilisateur demande la liste de ses favoris, le serveur doit interroger MongoDB puis TMDB. Ce modèle concurrent permet au backend de traiter plusieurs requêtes en parallèle sans bloquer le serveur principal.

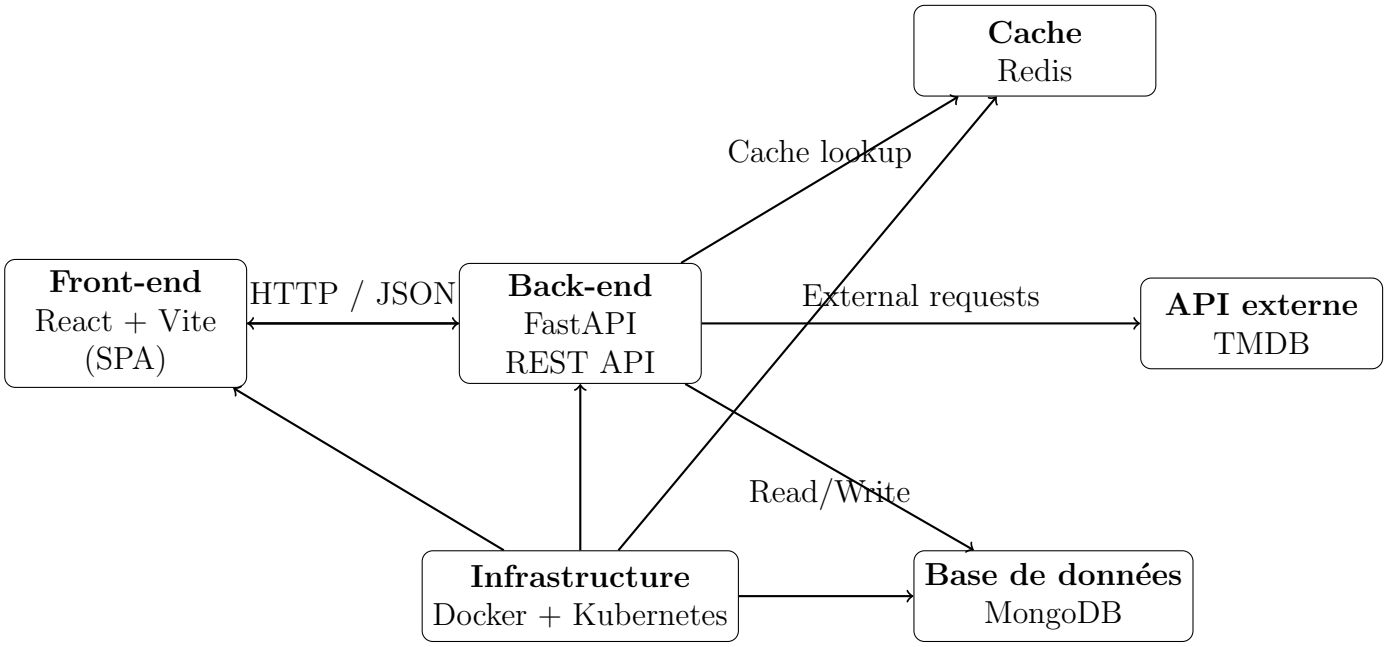


FIGURE 1 – Architecture globale du projet IShowCinema

3 Choix Techniques (Stack)

3.1 Étude et Choix de la Stack Back-end

Avant d'implémenter notre API, nous avons comparé trois langages et frameworks populaires en entreprise :

Etude de 3 langages back-end

- **Python :**
  - **Contextes d'utilisation :** Largement utilisé dans les domaines de la data science, de l'intelligence artificielle, ainsi que pour le développement d'API web via des frameworks comme FastAPI ou Django.
  - **Types d'applications :** API REST, Applications web backend, Machine learning / data engineering, Scripts d'automatisation

- **JavaScript (Node.js) :**
  - **Contextes d'utilisation :** Node.js est utilisé pour développer des applications web temps réel et des API backend en JavaScript côté serveur.
  - **Types d'applications :** API REST / GraphQL, Applications temps réel (chat, streaming), Microservices, Applications full-stack JavaScript

- **Go (Golang) :**
  - **Contextes d'utilisation :** Go est un langage conçu par Google pour les systèmes distribués, les microservices et les applications nécessitant de hautes performances.
  - **Types d'applications :** Microservices, APIs haute performance, Cloud computing, Systèmes distribués

KPI	Python (FastAPI)	Node.js (Express)	Go (Fiber)
Courbe d'apprentissage	Facile	Moyenne (si maîtrise de JS)	Moyenne
Productivité	Très élevée	Élevée	Moyenne
Performance	Moins adaptée aux traitements intensifs	Excellente pour I/O, limitée pour CPU	Excellente
Écosystème	Très riche	Très riche	Limité
Marché de l'emploi	Très forte (IA, data)	Très forte (web)	Forte (spécialisée)
Scalabilité	Bonne	Bonne	Excellente

Table 1 – Comparatif des langages Back-end selon nos KPI

Choix et Justification : Python et FastAPI

Nous avons retenu **Python couplé au framework FastAPI** pour l'implémentation de notre backend. Ce choix repose sur un compromis entre productivité, simplicité de développement et performance suffisante pour notre cas d'usage.

FastAPI supporte nativement la programmation asynchrone (`async/await`), ce qui permet d'effectuer des appels non bloquants vers des services externes tels que l'API TMDB. Cette approche est particulièrement adaptée à notre application, dont une partie des fonctionnalités dépend d'appels réseau fréquents.

De plus, FastAPI intègre automatiquement une documentation interactive via Swagger UI, facilitant la compréhension, le test et la maintenance de l'API.

Enfin, comparé à des alternatives comme Node.js ou Go, Python offre une productivité plus élevée pour le développement rapide de fonctionnalités, tout en conservant une architecture suffisamment performante pour notre besoin.

3.2 L'Environnement Virtuel (venv)

Pour le développement de notre back-end, nous avons mis en place un environnement virtuel (`venv`). Un environnement virtuel permet d'isoler les dépendances Python d'un projet afin d'éviter les conflits de versions entre plusieurs applications installées sur une même machine.

— **Intérêt en entreprise :** Sans environnement virtuel, une mise à jour d'une bibliothèque utilisée par un projet peut entraîner des incompatibilités avec un autre projet. La `venv` garantit ainsi la reproductibilité de l'environnement de développement, notamment via le fichier `requirements.txt`.

3.3 Étude et Choix de la Stack Front-end

Afin de développer une interface utilisateur moderne, rapide et maintenable, nous avons étudié plusieurs solutions front-end couramment utilisées en entreprise, notamment React, Angular et Vue.js. Notre choix s'est porté sur **React couplé à Vite**.

Etude des frameworks front-end

— **React :**
  — **Contextes d'utilisation :** React est une bibliothèque JavaScript largement utilisée pour la création d'interfaces utilisateur dynamiques et interactives, notamment dans les applications web modernes à forte composante front-end.
  — **Types d'applications :** Applications web SPA (Single Page Applications), dashboards, interfaces interactives, applications full-stack (MERN/MEAN stack).
— **Angular :**
  — **Contextes d'utilisation :** Framework complet développé par Google, utilisé dans les applications d'entreprise nécessitant une architecture fortement structurée.
  — **Types d'applications :** Applications d'entreprise complexes, systèmes internes, grandes applications web.
— **Vue.js :**
  — **Contextes d'utilisation :** Framework progressif utilisé pour développer des interfaces utilisateur simples à intermédiaires avec une prise en

main rapide.
— **Types d'applications :** Applications web légères à moyennes, prototypes, dashboards simples.

Comparaison des frameworks front-end selon des KPI

KPI	**React**	**Angular**	**Vue.js**
Courbe d'apprentissage	Moyenne	Difficile	Facile
Productivité	Élevée	Moyenne	Élevée
Performance	Très bonne	Bonne	Très bonne
Écosystème	Très riche	Très riche	Riche
Marché de l'emploi	Très forte	Très forte	Bonne
Flexibilité	Très élevée	Faible	Élevée
Scalabilité	Bonne	Excellente	Moyenne

TABLE 2 – Comparatif des frameworks front-end selon nos KPI

Choix et Justification : React et Vite

Nous avons retenu **React associé à Vite** pour le développement du front-end de notre application. React permet de construire des interfaces utilisateur modulaires basées sur des composants réutilisables, facilitant la maintenance et l'évolution du projet.

Le choix de Vite comme outil de build s'explique par ses performances supérieures aux outils traditionnels tels que Webpack. Vite offre un démarrage quasi instantané du serveur de développement et un rechargement très rapide, améliorant significativement l'expérience de développement.

De plus, React bénéficie d'un écosystème très riche et d'une forte adoption en entreprise, ce qui garantit la pérennité de la solution et facilite l'intégration dans des architectures modernes de type API REST.

Enfin, cette combinaison React + Vite offre un excellent compromis entre performance, productivité et maintenabilité, tout en restant adaptée aux architectures découplées front-end / back-end utilisées dans notre projet.

4 Données et API

4.1 Comprendre les API HTTP

Notre back-end agit comme une **API (Application Programming Interface)** permettant la communication entre le front-end (React) et le back-end (FastAPI) selon un modèle client-serveur.

Dans le cadre du web, les API HTTP reposent sur plusieurs concepts fondamentaux :

- **Endpoints :** points d'accès exposés par le serveur (ex : `/movies`, `/favorites`).
- **Méthodes HTTP :** elles définissent l'action à effectuer. Par exemple, `GET` permet de récupérer des données tandis que `POST` permet d'en envoyer.
- **Codes de statut HTTP :** ils indiquent le résultat de la requête (ex : 200 succès, 404 ressource introuvable, 500 erreur serveur).

Ces API sont dites **stateless**, car chaque requête contient toutes les informations nécessaires à son traitement.

4.2 Formats de Données : Le choix du JSON

Les échanges de données entre le front-end, le back-end et l'API TMDB utilisent le format **JSON (JavaScript Object Notation)**.

- **Définition :** Le JSON est un format léger basé sur des paires clé-valeur permettant de représenter des structures de données simples ou imbriquées (ex : {`"title": "Inception", "year": 2010`}).
- **Avantages :** Il est léger, lisible et facilement exploitable côté front-end via JavaScript.

Comparaison avec d'autres formats

Nous avons comparé JSON avec d'autres formats de données :

- **XML :** format structuré, nécessitant des balises ouvrantes et fermantes pour chaque élément, ce qui augmente la taille des données.
- **CSV :** adapté aux données tabulaires simples, mais limité pour représenter des structures complexes et hiérarchiques.

4.3 Autorisation des accès entre Front-end et Back-end : CORS

Lors du développement de notre application, nous avons été confrontés à un mécanisme de sécurité du navigateur appelé **CORS (Cross-Origin Resource Sharing)**.

Compréhension du CORS et Problème rencontré

CORS est un mécanisme de sécurité implémenté par les navigateurs web qui contrôle les requêtes HTTP effectuées depuis une origine différente de celle du serveur. Une origine est définie par la combinaison `protocole + domaine + port`. Dans notre cas, le front-end est exécuté sur `http://localhost:5173` (Vite), tandis que le back-end est exécuté sur `http://127.0.0.1:8000` (FastAPI). Bien que les deux applications soient locales, elles sont considérées comme deux origines différentes. Par défaut, le navigateur bloque ces requêtes pour éviter des attaques de type *Cross-Site Request Forgery (CSRF)*. Nos requêtes front-end vers l'API FastAPI étaient donc bloquées.

Solution mise en place et Résultat

Pour résoudre ce problème, nous avons utilisé le middleware CORS intégré à FastAPI en spécifiant les origines autorisées (l'URL de notre front-end), les méthodes HTTP et les en-têtes acceptés. Après configuration, la communication entre le front-end React et le back-end FastAPI a fonctionné correctement, permettant des échanges sécurisés.

4.4 Intégration d'une API réelle (TMDB) et rôle de Proxy

Dans ce projet, nous avons intégré une API externe réelle : **The Movie Database (TMDB)**.

Conformément aux bonnes pratiques de l'industrie, notre Front-end n'interroge jamais TMDB directement. C'est notre Back-end FastAPI qui agit comme un **Proxy** (ou API Gateway). Ce choix se justifie pour trois raisons :

— **Sécurité (Authentification) :** L'accès à l'API TMDB nécessite une clé d'authentification. Dans notre projet, cette clé est stockée côté serveur dans un fichier `.env` afin d'éviter toute exposition côté client.

— **Normalisation :** TMDB renvoie un volume de données très important (titre, acteurs, date, chemin du poster, etc.). Notre Back-end filtre ces données pour ne renvoyer au Front-end que l'essentiel, réduisant ainsi la charge réseau.
— **Gestion des erreurs :** Le Back-end intercepte les codes d'erreur de TMDB et renvoie un message d'erreur propre à l'interface utilisateur.

4.5 Choix de la source de données : API externe vs Dataset existant

Lors de la conception de notre catalogue de films, nous avons dû choisir entre un jeu de données statiques et une source dynamique. Notre choix s'est porté sur l'API TMDB pour plusieurs raisons :

— **Données en temps réel :** L'utilisation de l'API TMDB nous permet d'afficher les films actuellement en salle, les tendances du jour et des informations toujours à jour, offrant ainsi une véritable expérience utilisateur.
— **Problématiques concrétes :** De plus, ce choix nous a permis de travailler sur des problématiques concrètes telles que :
  — l'authentification via une clé API ;
  — la gestion des requêtes HTTP et des erreurs ;
  — la normalisation des données reçues ;
— **Espace de stockage :** L'API nous évite de devoir stocker nous-mêmes toutes les données et les images des films.

Limites de cette approche

Malgré ses avantages, l'utilisation d'une API externe présente certaines contraintes :
— dépendance à un service tiers ;
— limitations du nombre de requêtes (*rate limiting*) ;
— nécessité de sécuriser la clé API côté serveur.

4.6 Collecte de données : Export vs Web Scraping

Dans le cadre de ce projet, nous avons implémenté une fonctionnalité d'export permettant de sauvegarder une liste de films au format JSON. Cette

collecte a soulevé une réflexion sur les méthodes d'extraction sur le web. Nous avons privilégié l'utilisation de l'API officielle plutôt que le **Web Scraping** (l'extraction automatisée depuis le HTML des pages web). L'approche API constitue la voie officielle : les conditions d'utilisation sont claires et les limites de requêtes sont encadrées. À l'inverse, le Web Scraping pose des défis éthiques et techniques.

4.7 Bonnes pratiques avancées en production

Si notre application devait être déployée pour des milliers d'utilisateurs, plusieurs mécanismes d'optimisation seraient indispensables :

- **Rate Limiting :** Implémenter un algorithme de *Backoff exponentiel* pour éviter que notre serveur ne soit banni par TMDB en cas de pic d'affluence.
- **Pagination dynamique :** Mettre en place un système de *défilement infini* côté Front-end pour naviguer dans le catalogue sans surcharger la mémoire du navigateur.

5 Stockage et Performance

La gestion des données est un enjeu critique dans une architecture. Pour répondre aux besoins de persistance d'une part, et de rapidité d'exécution d'une autre, nous avons opté pour une approche hybride en utilisant deux technologies distinctes : MongoDB et Redis.

5.1 Persistance des données avec MongoDB

Pour stocker de manière permanente les comptes utilisateurs et leurs listes de films favoris, nous avons choisi **MongoDB**, un système de gestion de base de données (SGBD) orienté documents (NoSQL).

Justification du choix NoSQL

Contrairement à une base de données relationnelle classique (comme PostgreSQL ou MySQL) qui stocke les données dans des tables rigides, MongoDB stocke les informations sous forme de documents BSON (une version binaire du JSON). Ce choix s'inscrit dans une continuité technique parfaite avec le reste de notre architecture :

- Notre Front-end (React) consomme du JSON.
- Notre Back-end (FastAPI) traite du JSON.
- L'API externe (TMDB) renvoie du JSON.

L'utilisation de MongoDB nous permet de sauvegarder directement les structures de données sans avoir à créer de lourds mécanismes de mapping. De plus, l'absence de schéma strict offre une grande flexibilité si nous décidons d'ajouter de nouveaux champs au profil utilisateur à l'avenir.

5.2 Optimisation et Cache avec Redis

L'un des défis majeurs de l'utilisation d'une API externe comme TMDB est la latence réseau (le temps d'aller-retour de la requête HTTP) et les quotas d'appels imposés par la plateforme (*Rate Limiting*). Pour pallier ces problèmes, nous avons intégré **Redis**, un magasin de structures de données en mémoire vive (RAM), utilisé ici comme système de cache.

Implémentation du pattern Cache-Aside

Nous avons implémenté la stratégie de *Cache-Aside* (ou *Lazy Loading*) pour optimiser la route affichant les films populaires :

- **Étape 1 (Vérification) :** Lorsqu'un utilisateur demande la liste des films, le back-end vérifie d'abord si la réponse est déjà stockée dans Redis.
- **Étape 2 (Dans le cache) :** Si les données sont présentes en mémoire, elles sont renvoyées instantanément au client, le temps de réponse est beaucoup plus court.
- **Étape 3 (Pas dans le cache) :** Si les données sont absentes, le back-end interroge TMDB, formate la réponse JSON, puis la sauvegarde dans Redis avant de l'envoyer à l'utilisateur.

Cette stratégie permet de réduire significativement le nombre de requêtes vers TMDB et d'améliorer les temps de réponse de l'application.

Gestion des changements (TTL)

Pour garantir que notre catalogue ne propose pas des films obsolètes, nous avons configuré un **TTL (Time To Live)** sur les clés Redis. Par exemple, le cache des films peut expirer toutes les heures. Une fois le TTL écoulé, Redis supprime automatiquement la donnée, forçant le back-end à refaire un appel réseau vers TMDB lors de la prochaine requête, garantissant ainsi la fraîcheur des données.

6 Conteneurisation et Orchestration

La conception d'une architecture orientée distribution n'a de sens que si elle s'accompagne d'une infrastructure capable de la déployer, de la maintenir et de la rendre scalable. Pour cela, nous avons intégré **Docker** pour la conteneurisation et **Kubernetes** (via Minikube) pour l'orchestration.

6.1 Isolation des services avec Docker

La première étape de notre démarche DevOps a consisté à isoler chaque brique de notre application dans son propre conteneur Docker.

Création des images (Dockerfiles)

Nous avons rédigé deux fichiers `Dockerfile` distincts :

- **Image Front-end (React) :** Basée sur une image Node.js légère, elle installe les dépendances via `npm`, construit l'application avec Vite, puis expose le port statique.
- **Image Back-end (FastAPI) :** Basée sur une image Python optimisée (`python:3.10-slim`), elle copie le fichier `requirements.txt`, installe les bibliothèques dans un environnement standardisé, et lance le serveur ASGI Uvicorn.

Justification de la conteneurisation

L'utilisation de Docker résout le célèbre problème du "Ça marche sur ma machine". En encapsulant le code, les dépendances et le système d'exploitation de base dans une image immuable, nous garantissons que l'application s'exécutera de la même manière en développement (local), en test et en production. De plus, si l'image du front-end venait à crasher, cela n'impacterait en rien le fonctionnement du back-end.

6.2 Orchestration et Haute Disponibilité avec Kubernetes

Si Docker permet de créer des conteneurs, il ne suffit pas pour gérer une infrastructure de production complexe (redémarrage en cas de panne,

réseau interne, montée en charge). C'est pourquoi nous avons déployé notre application sur un cluster local **Kubernetes** (Minikube).

Notre déploiement s'articule autour de trois concepts clés de Kubernetes :

1. Deployments et Haute Disponibilité (Pods)

Au lieu de lancer de simples conteneurs isolés, nous avons défini des **Deployments**. Le composant le plus critique de notre infrastructure étant le Back-end (FastAPI), nous l'avons configuré avec plusieurs **réplicas** (`replicas: 2`). Ainsi, Kubernetes répartit la charge entre deux instances (Pods) distinctes. Si l'un des Pods tombe en panne à cause d'une erreur fatale, Kubernetes applique son mécanisme de *Self-Healing* (auto-guérison) et redémarre instantanément un nouveau Pod pour le remplacer, garantissant une véritable **Haute Disponibilité**.

2. Le Réseau Interne (Service Discovery)

Dans Kubernetes, les Pods sont éphémères et leurs adresses IP changent constamment. Pour permettre au Front-end de communiquer avec le Back-end, et au Back-end de communiquer avec MongoDB, nous avons créé des ressources de type **Service**. Le *Service Discovery* permet à nos composants de s'appeler par leur nom de domaine interne (ex : `mongodb-service`) plutôt que par une adresse IP figée. Cela rend notre architecture totalement robuste face aux redémarrages.

3. Persistance des Données (Persistent Volumes)

Par définition, un conteneur est dit "stateless" et supprime toutes ses données à sa destruction. Or, notre base de données MongoDB se doit d'être "stateful" (avec état). Pour éviter de perdre les comptes utilisateurs et les favoris à chaque redémarrage du cluster, nous avons configuré un **PersistentVolumeClaim (PVC)**. Ce mécanisme demande à Kubernetes de réserver un espace de stockage sur le disque dur de la machine hôte et de le "brancher" au Pod MongoDB. Ainsi, le cycle de vie de la donnée est dissocié du cycle de vie du conteneur.

7 CI/CD et Reproductibilité

Dans une démarche DevOps, le développement et le déploiement ne doivent plus dépendre d'actions manuelles, souvent sources d'erreurs. Pour garantir la fiabilité de notre application IShowCinema, nous avons mis en place des pratiques d'intégration continue (CI) et assuré la reproductibilité totale de notre infrastructure.

7.1 L'Infrastructure as Code (IaC)

L'un des principes fondamentaux de la reproductibilité est l'**Infrastructure as Code**. Plutôt que de configurer nos bases de données, notre réseau ou nos serveurs manuellement, l'intégralité de notre infrastructure est définie dans des fichiers déclaratifs (nos manifestes YAML pour Kubernetes et nos `Dockerfile`). Ces fichiers étant versionnés sur Git avec notre code source, n'importe quel développeur clonant le dépôt obtient exactement la même architecture technique, garantissant une reproductibilité parfaite.

7.2 Automatisation avec une Pipeline CI/CD

Pour automatiser le cycle de vie de notre code, nous avons conçu une pipeline d'intégration et de déploiement continus (via GitLab CI/CD). Cette pipeline se déclenche automatiquement à chaque nouveau *commit* ou *Merge Request* et se divise en plusieurs étapes (stages) :

- **Stage "Test" (Intégration Continue) :** Avant d'accepter du nouveau code, la pipeline vérifie la syntaxe (ex : utilisation de `Flake8` pour le Python) et lance d'éventuels tests unitaires. Cela garantit que le nouveau code ne "casse" pas l'application existante.
- **Stage "Build" :** Si les tests passent, la pipeline construit les nouvelles images Docker pour le Front-end et le Back-end afin de s'assurer que la conteneurisation est toujours fonctionnelle.
- **Stage "Deploy" (Déploiement Continu) :** Dans un environnement de production réel, cette étape pousserait les images vers une *Container Registry* et mettrait à jour les Pods Kubernetes automatiquement.

7.3 Scripts de lancement

Dans le cadre de notre développement local sous Minikube, déployer l'application nécessite d'exécuter de nombreuses commandes (*build* des images, application des manifestes YAML, ouverture des tunnels réseau).

Pour éviter les erreurs humaines et simplifier la prise en main du projet par un tiers, nous avons regroupé toutes ces opérations dans un script d'automatisation (`start.sh`). Ce script exécute séquentiellement :

1. Le démarrage du cluster Minikube.
2. La construction des images Docker directement dans l'environnement du cluster.
3. L'application des manifestes de configuration (`kubectl apply -f ...`).
4. La mise en place des redirections de ports (`port-forward`) pour rendre le site accessible sur la machine hôte.

Grâce à ce script, l'ensemble de la plateforme (Front, Back, MongoDB, Redis) peut être déployé et rendu fonctionnel en une seule commande.

8 Conclusion, Difficultés et Perspectives

8.1 Bilan du projet

La réalisation du projet IShowCinema a été une expérience particulièrement enrichissante qui nous a permis de faire le pont entre le développement classique et les contraintes de l'ingénierie système. Nous avons réussi à concevoir une application fonctionnelle, interfaçant un Front-end dynamique (React) avec une API externe réelle (TMDB) via notre propre Proxy backend (FastAPI), tout en persistant les données (MongoDB) et en optimisant les performances (Redis). Le déploiement conteneurisé sous Kubernetes a concrétisé notre démarche d'infrastructure résiliente et scalable.

8.2 Difficultés techniques rencontrées

— **La machine virtuelle :** La machine virtuelle a constitué une source importante de difficultés lors du développement de l'application. En particulier, le manque d'espace disque a entraîné des problèmes fréquent notamment lors de la partie Docker et Kubernetes.

— **Kubernetes :** Des difficultés ont été rencontrées lors de la mise à jour de l'application. En effet, les modifications apportées au code du site n'étaient pas toujours prises en compte après redéploiement. Ce problème était principalement dû au cache des images Docker et au fonctionnement de Kubernetes, qui ne redéploie pas automatiquement les conteneurs lorsque le tag de l'image ne change pas. Cela a entraîné des incohérences entre le code développé et la version effectivement déployée.

8.3 Conclusion générale

En conclusion, le projet IShowCinema a été une expérience très formatrice. C'était la première fois que nous réunissions autant de compétences au sein d'un même projet. Il a constitué une véritable introduction aux pratiques DevOps, en partant de Docker à l'automatisation des tâches (CI/CD). Malgré les défis rencontrés lors du déploiement sous Kubernetes, nous avons réussi à concevoir une application pleinement fonctionnelle de bout en bout. Le dépassement de ces obstacles techniques nous a permis de consolider nos acquis et de mieux appréhender les exigences d'une architecture moderne.